

Sistema de Matchmaking

Luis Eduardo Weigert Weiss

Murilo Granemann de Souza

17 de maio de 2026

1 Implementação dos Algoritmos de Ordenação e Recursão

- **Insertion Sort:** Implementado em `sortByScoreInsertion()`. Percorre a estrutura linearmente a partir do segundo elemento ($i = 1$) e o posiciona retrogradamente entre os elementos já ordenados à esquerda. O deslocamento de dados ocorre em um laço `while` enquanto o jogador comparado possuir *score* maior ou, em caso de empate, um *timestamp* maior.
- **Merge Sort e Recursão:** Dividido entre `sortByScoreMerge()`, `mergeSort()` e `merge()`. O método `mergeSort()` divide o array sucessivamente ao meio via recursão até atingir os casos base (tamanho ≤ 1). As chamadas recursivas resolvem as duas metades e a função `merge()` realiza a intercalação linear dos subvetores gerando um array dinâmico auxiliar, cujos dados são copiados de volta para o vetor principal.
- **Critério de Desempate:** A estabilidade e o desempate cronológico são garantidos pelas condicionais de comparação. Quando dois jogadores apresentam o mesmo *score*, o jogador com o menor *timestamp* (ordem de chegada) é posicionado obrigatoriamente antes na fila.

2 Aproveitamento da Ordenação para Formação de Grupos

- **Uso da Fila Ordenada:** O método `formGroup(groupSize, delta, n)` assume que a fila de jogadores está ordenada por *score*. Isso garante que jogadores com níveis de habilidade próximos estejam em posições consecutivas, eliminando buscas combinatórias complexas.
- **Janela Deslizante:** A busca por um grupo válido é realizada por uma varredura linear utilizando uma janela de tamanho fixo equivalente a `groupSize` (M). Como os scores estão crescentes, o menor valor da janela está no primeiro índice (i) e o maior no último ($i + M - 1$). A validação do critério é feita em tempo constante $O(1)$ avaliando apenas as extremidades da janela: `players[i + M - 1].getScore() - players[i].getScore() ≤ delta`. O método extrai o primeiro grupo válido encontrado, copia-o para o array de retorno e remove os jogadores da estrutura original aplicando um deslocamento à esquerda (*shift*) nos elementos restantes.

3 Custo Computacional

Considerando N como o número de jogadores ativos aguardando na fila de espera do sistema:

- **Inserção (insert):** $O(1)$. O novo jogador é inserido imediatamente na posição subsequente ao final do array controlado pelo atributo `size`, sem necessidade de deslocamentos.
- **Remoção (removePlayer):** $O(N)$. Realiza uma busca linear sequencial para localizar o *ID* e executa o rearranjo por deslocamento de todos os elementos à direita para cobrir a lacuna aberta.
- **Insertion Sort:** $O(N^2)$. Possui dois laços aninhados e, no pior caso (vetor inversamente ordenado), realiza um número quadrático de comparações e movimentações.
- **Merge Sort:** $O(N \log N)$. A árvore recursiva possui profundidade $\log N$, e cada nível executa uma operação de intercalação (*merge*) de custo estritamente linear $O(N)$.
- **Formação de Grupo (formGroup):** $O(N)$. A varredura da janela deslizante para localizar o grupo consome no máximo $O(N)$ passos. Encontrado o grupo, a compactação do array via *shift* também custa $O(N)$.
- **Recuperação (getWaitingPlayers):** $O(N)$. Percorre linearmente a fila para copiar sequencialmente os dados de cada jogador para um novo vetor alocado dinamicamente no heap.

4 Análise Experimental de Desempenho

- **Metodologia dos Testes:** Os tempos foram medidos com amostras de tamanhos $N \in \{100, 500, 1000, 5000, 10000\}$ na função `testDesempenho()`. Para garantir repetibilidade e uma comparação justa, os dados foram gerados deterministicamente pela fórmula desordenada $f(i) = (i \times 13) \pmod{5000}$. A cronometragem foi realizada em milissegundos via biblioteca `<chrono>`.
- **Tabela de Tempos Medidos:**

Quantidade de Jogadores (N)	Insertion Sort (ms)	Merge Sort (ms)
100	0.026	0.388
500	2.005	0.610
1000	9.655	2.185
5000	208.573	7.862
10000	855.332	16.275

Tabela 1: Comparação empírica de tempo entre os algoritmos de ordenação.

- **Discussão e Complexidade:** Os resultados práticos confirmam as curvas assintóticas teóricas. Para $N = 100$, o *Insertion Sort* é ligeiramente mais rápido por não sofrer com o overhead de chamadas recursivas e alocações dinâmicas. Contudo, em volumes maiores, o comportamento quadrático do *Insertion Sort* degrada o desempenho rapidamente (o tempo quadruplica quando a entrada dobra de 5000 para 10000). O *Merge Sort* mantém o crescimento quase-linear de ordem $O(N \log N)$, mostrando-se massivamente superior e eficiente em larga escala.